Szkoła Tańca

Dokumentacja projektowa PZSP2

Wersja 1

19 stycznia 2026

Semestr 25Z

Zespół nr 2 w składzie:

Gavrylenko Ihor
Kukiełka Wojciech
Rybachok Andrii
Yedzeika Sofiya

Mentor zespołu: dr inż. Marcin Szlenk
Właściciel tematu: dr inż. Żółtowska Izabela

Spis treści

1 Wprowadzenie **4**
1.1 Cel projektu 4
1.2 Wstępna wizja projektu 4

2 Metodologia wytwarzania **4**

3 Analiza wymagań **5**
3.1 Wymagania użytkownika i biznesowe 5
3.2 Wymagania funkcjonalne i niefunkcjonalne 6
3.3 Przypadki użycia 7
3.4 Potwierdzenie zgodności wymagań 9

4 Definicja architektury **9**
4.1 Warstwy i komponenty systemu 9
4.2 Podział backendu na moduły 10
4.3 Zastosowane szablony architektoniczne 11
4.4 Interfejsy kluczowych elementów oraz interakcje 11
4.5 Perspektywa logiczna/funkcjonalna 13
4.6 Perspektywa sprzętowa (deployment) i środowisko uruchomieniowe 13
4.7 System bazy danych i mechanizmy utrwalania danych 14
4.8 Raportowanie oraz analityka/BI 14
4.9 Mechanizmy zarządzania 14
4.10 Mechanizmy bezpieczeństwa 14

5 Dane trwałe **15**
5.1 Model logiczny danych 15
5.2 Przetwarzanie i przechowywanie danych 15

6 Specyfikacja analityczna i projektowa **16**
6.1 Metody realizacji projektu 16
6.2 Statystyki 16

7 Projekt standardu interfejsu użytkownika **17**
7.1 Narzędzia 17
7.2 Wpływ prototypu na gotowy produkt 18

8 Specyfikacja testów **18**
8.1 Standardy jakości 18
8.2 Testy jednostkowe 18
8.2.1 Sytuacje wyjątkowe 18
8.2.2 Obsługa sytuacji wyjątkowych 19
8.2.3 Automatyczne testy interfejsu 19
8.2.4 Miary jakości testów 19

9 Wirtualizacja / konteneryzacja **19**

10 Bezpieczeństwo **19**
10.1 Uprawnienia użytkowników 19

11 Podręcznik użytkownika **20**
11.1 Uczeń 20
11.2 Instruktor 20
11.3 Administrator 20

12 Podręcznik administratora **20**
12.1 0) Informacje ogólne o systemie 20
12.2 1) Instrukcja budowy systemu z kodu źródłowego 21
12.2.1 1.1 Wariant zalecany: Docker Compose 21
12.2.2 1.2 Uruchomienie lokalne (bez Dockera) – głównie do developmentu 22
12.3 2) Instrukcja instalacji i konfiguracji systemu 22
12.3.1 2.1 Konfiguracja backendu (Django) 22
12.3.2 2.2 Konfiguracja reverse proxy (Nginx) i HTTPS 23
12.3.3 2.3 Konfiguracja frontendu (API URL) 23
12.4 3) Instrukcja aktualizacji oprogramowania 24
12.4.1 3.1 Aktualizacja w Docker Compose (typowa procedura) . 24
12.4.2 3.2 Aktualizacja frontendu 24
12.4.3 3.3 Rollback (procedura awaryjna – skrót) 24
12.5 4) Instrukcja zarządzania użytkownikami i uprawnieniami 24
12.5.1 4.1 Panel administracyjny Django 24
12.5.2 4.2 Role i logika uprawnień 25
12.5.3 4.3 Uwierzytelnianie (JWT) 25
12.5.4 4.4 Operacje administracyjne na kontach (przykłady) . . 25
12.6 5) Instrukcja tworzenia kopii zapasowych i odtwarzania systemu . 26
12.6.1 5.1 Backup bazy danych (PostgreSQL) 26
12.6.2 5.2 Backup wolumenu (alternatywnie) 26
12.6.3 5.3 Backup konfiguracji i certyfikatów 26
12.7 6) Instrukcja zarządzania zasobami systemu 26
12.7.1 6.1 Operacje na usługach (Docker) 26
12.7.2 6.2 Monitorowanie dostępności (healthcheck) 27
12.7.3 6.3 Wydajność i limity 27
12.7.4 6.4 Kluczowe endpointy operacyjne (skrót) 27

13 Podsumowanie **28**
13.1 Czego się nauczyliśmy? 28
13.2 Dalsze kroki 28

1 Wprowadzenie

1.1 Cel projektu

Celem jest stworzenie aplikacji wspomagającej właścicielkę Szkoły Tip Tap w zarządzaniu ofertą oraz w realizacji bieżących zadań organizacyjnych. Konkretnie, system ma za zadanie wspomóc właścicielkę i instruktorów przy realizacji lekcji stepowania, a uczniom ma uprościć formalności związane z uczęszczaniem na te zajęcia.
Załączony został dokument SZKOŁA TAŃCA.DOCX, zawierający wymagania właścicielki.

1.2 Wstępna wizja projektu

Sercem systemu będzie aplikacja webowa, która będzie komunikować się z serwerem aplikacyjnym. Będzie stworzona baza danych. W aplikacji będą trzy rodzaje użytkowników: uczniowie, instruktorzy i administrator. Podczas projektowania będzie kładziony nacisk na łatwość obsługi, automatyzację czynności i bezpieczeństwo danych osobowych.

2 Metodologia wytwarzania

Nasz zespół składa się z 4-ech osób. Mamy mentora, z którym będziemy się spotykać co około 3 tygodnie. Stosujemy podejście zwinne, więc jego rolą nie jest zarządzanie, lecz wsparcie merytoryczne.
Sami w swojej strukturze też nie mamy przewodniczącego, który przydziela zadania, aczkolwiek mamy lidera – Ihora, który jest z nas najbardziej doświadczony. Każdy członek zespołu jest zmotywowany powodzeniem projektu i sam podejmuje się zadań, biorąc pod uwagę swoje siły i kompetencje.

Schemat pracy

Głównym spoiwem zespołu są cotygodniowe spotkania, na których przedstawiamy postępy, dzielimy się spostrzeżeniami i określamy dalsze kierunki działania. Poza spotkaniami pracujemy, samodzielnie lub w parach.

Role Kluczowe jest, aby co najmniej dwie osoby potrafiły zrobić dowolną czynność deweloperską bądź administracyjną. Chcąc zaspokoić to oczekiwanie, zadbaliśmy o to, żeby równomiernie przypisać ludzi do odpowiedzialności. Dokładnie, projekt ma 3 komponenty – bazę danych, front-end i back-end. Każdym komponentem zajmują się co najmniej dwie osoby.

Narzędzia Podstawą jest narzędzie kontroli wersji – *git*. Oprócz tego potrzebne jest miejsce do określania zadań. Obie te funkcje ma platforma *gitlab*.

Mówiąc bardziej technicznie, jednostka pracy (zadanie, issue) będzie odnotowywane w *gitlabie*, wolontariusz przypisze to zadania do siebie do wykonania, utworzy niezależną gałąź z kodem, zaimplementuje, co trzeba, a następnie oznaczy gałąź, jako gotową do wdrożenia do oficjalnej wersji kodu. Przed wdrożeniem, gałąź będzie musiała przejść, w pierwszej kolejności – testy automatyczne, a następnie recenzję innego członka zespołu.

3 Analiza wymagań

3.1 Wymagania użytkownika i biznesowe

Wymagania biznesowe

B1 System **musi** odciążyć właścicielkę w rejestracji uczniów, ich rozliczaniu oraz w kontakcie z nimi.

B2 Aplikacja **musi** pozwolić rejestrować obecności uczniów.

B3 Aplikacja **musi** pozwolić uczniom zarejestrować się na zajęcia.

B4 Aplikacja **musi** wyposażyć uczniów w informacje dotyczące ich uczestnictwa w zajęciach, aby, w szczególności, dotrzymywanie terminów płatności było dla nich łatwiejsze.

B5 Aplikacja **musi** pozwolić właścicielce zmieniać ceny karnetów oraz wprowadzać inne zmiany istotne z punktu widzenia systemu.

B6 Aplikacja **powinna** pozwalać na tworzenie raportów dotyczących sprzedaży karnetów.

Wymagania użytkownika

Powyższe wymagania pociągają za sobą poniższe wymagania użytkownika.

U1 Uczeń **musi** móc się zarejestrować i zalogować adresem e-mail.

U2 Rejestracja **powinna** wymagać zatwierdzenia automatycznego e-maila.

U3 Pomyślne zarejestrowanie się **musi** być jasno zakomunikowane, w szczególności komunikat o tym zdarzeniu powinien być trudny do przeoczenia.

U4 Uczeń **musi** móc zapisać się na zajęcia.

U5 Uczeń **powinien** móc wpisać się na listę rezerwową.

U6 Uczeń **musi** widzieć status swoich płatności

U7 Uczeń **musi** widzieć ile przysługuje mu odrobień.

U8 Dwaj uczniowie **muszą** móc mieć ten sam e-mail.

U9 Uczeń **musi** móc usunąć swoje konto, prosząc o usunięcie jego danych z systemu.

I1 Instruktor **musi** móc odnotować obecność ucznia na zajęciach.

I2 Instruktor **musi** móc odnotować obecność ucznia odrabiającego zajęcia.

I3 Instruktor **musi** móc skorygować obecność ucznia do 7 dni po zajęciach.

I4 Instruktor **musi** mieć dostęp **tylko** do niezbędnych danych, w szczególności **nie może** mieć dostępu do e-maili.

Uwaga: właścicielka jest administratorem.

A1 Administrator **musi** móc modyfikować grafik zajęć.

A2 Administrator **musi** móc dodawać konta instruktorów.

A3 Administrator **powinien** móc modyfikować dane uczniów.

A4 Administrator **powinien** móc wygenerować podsumowanie płatności z karnetów za dany okres.

A5 Administrator **musi** móc odnotować płatność ucznia.

A6 Administrator **powinien** móc zapisać płatność *w przód,* np. uczeń opłaca za dwa miesiące.

A7 Administrator **musi** mieć dostęp do danych ucznia, który ma nieuregulowane płatności, nawet jeśli usunął konto.

A8 Administrator **powinien** móc wysłać maila do całej grupy zajęciowej.

3.2 Wymagania funkcjonalne i niefunkcjonalne

Wszystkie powyższe wymagania oprócz **U3** są przeważnie funkcjonalne, lecz oczywiste jest, że aby je zrealizować, trzeba spełnić kilka wymagań niefunkcjonalnych.

N1 Interfejs **musi** być prosty, czytelny oraz mieć komunikaty o sukcesach/błędach.

N2 Systemu **musi** być dostępny całodobowo z poziomu przeglądarki.

N3 Interfejs **musi** być równie dobry na urządzeniach mobilnych.

N4 Dane osobowe **muszą** być przechowywane bezpiecznie z poszanowaniem RODO.

N5 System **musi** być utrzymywalny.

Mapowania wymagań

	U1	U2	U3	U4	U5	U6	U7	U8	U9	I1	I2	I3	I4	A1	A2	A3	A4	A5	A6	A7	A8
B1																		■	■	■	■
B2										■	■	■	■								
B3				■	■				■												
B4	■	■	■			■	■	■													
B5														■	■	■					
B6																	■				

3.3 Przypadki użycia

PB1 Rejestracja ucznia

1. Uczeń wypełnia formularz rejestracyjny.
2. System wysyła e-mail weryfikacyjny.
3. Uczeń potwierdza adres e-mail.
4. System aktywuje konto i potwierdza rejestrację.

PB2 Zapisanie się na zajęcia

1. Uczeń przegląda dostępne terminy zajęć.
2. Uczeń wybiera termin.
3. Uczeń potwierdza zapis.
4. System sprawdza, czy miejsce jest wolne.
   (a) Jeśli tak, zapis jest rejestrowany w systemie.
   (b) Jeśli nie, uczeń jest powiadamiany o błędzie.

PB3 Przegląd informacji o koncie ucznia

1. Uczeń klika na przycisk „Moje Konto”.
2. Wyświetla się ogólna informacja o koncie: imię, nazwisko itp.
3. Menu zawiera następujące zakładki:
   (a) Lista zajęć, na które uczeń jest zapisany
   (b) Dziennik obecności, gdzie uczeń może widzieć historię zajęć.
   (c) Zakładka „Płatności” z informacjami o płatnościach
4. Uczeń klika na jeden z przycisków, aby obejrzeć interesującą go informację.

PB4 Odrabianie nieobecności

1. Uczeń widzi w Dzienniku obecności (PB3), że ma do odrobienia nieobecność.
2. Następnie przychodzi na zajęcia innej grupy.
3. Instruktor odnotowuje jego obecność w systemie.

PB5 Ewidencja obecności na zajęciach

1. Instruktor widzi listę zajęć prowadzonych w tym oraz poprzednim tygodniu.
2. Instruktor wybiera zajęcia i widzi listę uczestników tej grupy.
3. Instruktor oznacza obecności i dopisuje osoby odrabiające.

PB6 Korekta list obecności

1. Instruktor wybiera odpowiednie zajęcia.
2. Instruktor potwierdza, że chce **edytować** obecności.
3. Instruktor wprowadza korekty i opcjonalnie dopisuje odrabiających.

PB6 Usuwanie kont użytkowników

1. Użytkownik zgłasza chęć usunięcia konta.
   (a) Jeśli wszystka ma opłacone, konto wraz z danymi jest usuwane / anonimizowane.
   (b) Jeśli nie, do administratora wysyłana jest prośba o usunięcie konta.

1. Administrator wybiera w menu opcję „Zarządzanie kontami”.
2. Wyświetla się lista użytkowników oraz wyszukiwarka do znalezienia konkretnej osoby.
3. Administrator wybiera użytkownika do usunięcia (lub potwierdza zgłoszone usunięcie konta przez ucznia).
4. System deaktywuje konto.

PB7 Dodanie zajęć

1. Administrator wchodzi na panel zarządzania zajęciami.
2. Naciska przycisk „Dodaj zajęcia”.
3. Wyświetla się okienko do wypełnienia informacji o zajęciu.
4. Administrator potwierdza i tworzy zajęcie.
5. Od teraz uczniowie mogą zapisywać się na te zajęcia, a instruktor widzi je w terminarzu.

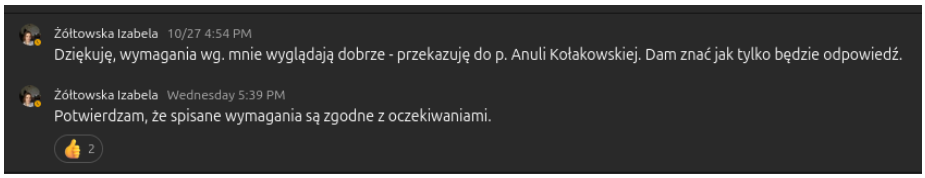
PB8 Modyfikacja zajęć

1. Na stronie zarządzania zajęciami jest wyświetlana lista wszystkich zajęć.
2. Administrator modyfikuje wybrany termin zajęć lub usuwa go.
3. Jeśli zajęcia są usuwane, system wypisuje uczniów i instruktora z tych zajęć.

PB9 Modyfikacja danych ucznia

1. Administrator wchodzi na stronę zarządzania użytkownikami.
2. Administrator wyszukuje konto użytkownika.
3. Naciska przycisk „Edytuj”.
4. Administrator koryguje dane.

3.4 Potwierdzenie zgodności wymagań



4 Definicja architektury

4.1 Warstwy i komponenty systemu

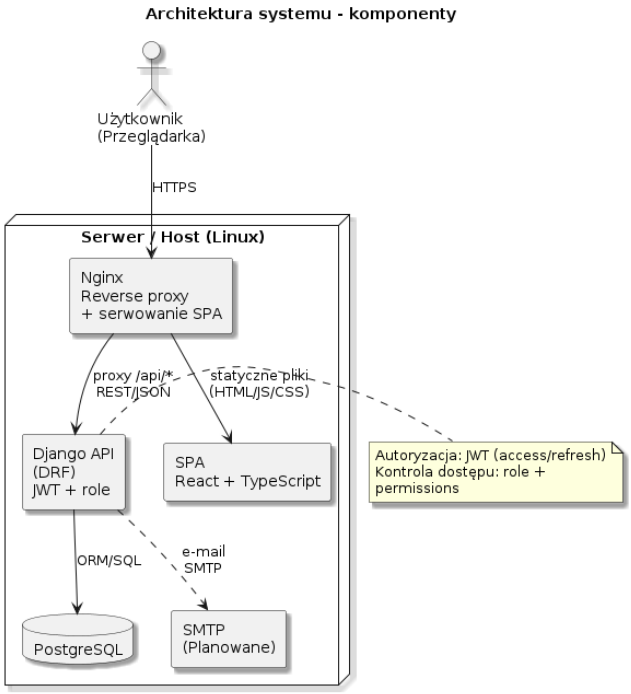
System ma architekturę trójwarstwową:

- warstwa prezentacji: aplikacja SPA (React + TypeScript),
- warstwa logiki biznesowej: backend Django + DRF (REST API),

- warstwa danych: relacyjna baza PostgreSQL.

W architekturze wdrożeniowej punkt wejścia stanowi Nginx. Pełni on dwie role:

- serwuje pliki statyczne aplikacji SPA,
- przekazuje zapytania do backendu pod ścieżką `/api/*`.



Rysunek 1: Architektura

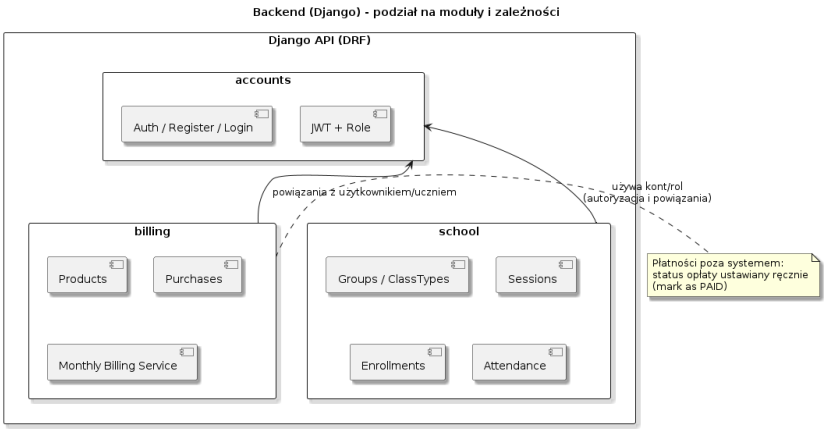
4.2 Podział backendu na moduły

Backend jest monolitem modularnym, podzielonym na trzy aplikacje domenowe:

- `accounts` – konta użytkowników, role, logowanie i tokeny JWT,
- `school` – grupy i typy zajęć, sesje zajęć, zapisy, obecności,
- `billing` – produkty, należności miesięczne (`Purchase`), ręczne oznaczanie wpłat jako opłaconych.

Dodatkowe elementy:

- SMTP – wysyłka e-mail (np. potwierdzenia/przypomnienia),
- zadania cykliczne – np. generacja rozliczeń i/lub przypomnienia.



4.3 Zastosowane szablony architektoniczne

W projekcie zastosowano:

- architekturę client–server (SPA jako klient, Django API jako serwer),
- komunikację REST/JSON pomiędzy SPA i backendem,
- monolit modularny po stronie backendu (moduły domenowe jako Django apps),
- podejście MVC/MVT: modele w ORM, logika w endpointach DRF (ViewSety/serwisy), prezentacja w SPA.

4.4 Interfejsy kluczowych elementów oraz interakcje

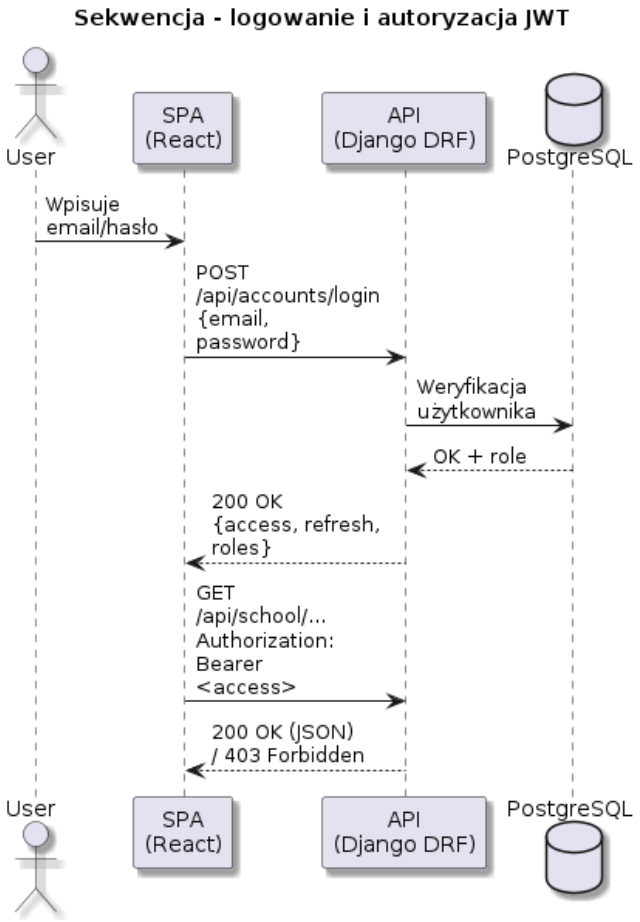
Komunikacja i interfejsy:

- HTTPS: przeglądarka ↔ Nginx,
- REST/JSON: SPA ↔ backend (ścieżka `/api/*`),
- JWT: nagłówek `Authorization: Bearer <token>`,
- SQL/ORM: backend ↔ PostgreSQL,
- (plan) SMTP: backend ↔ serwer pocztowy.

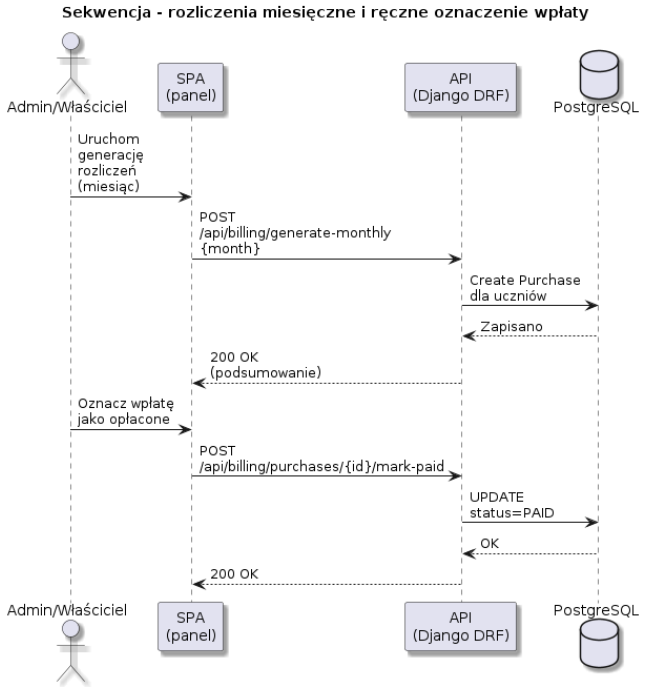
Najważniejsze scenariusze interakcji:

- logowanie użytkownika i autoryzacja żądań przez JWT,

- generacja rozliczeń miesięcznych i ręczne oznaczenie wpłaty jako opłaconej.



Rysunek 2: Sekwencja autoryzacji



Rysunek 3: Sekwencja rozliczania

4.5 Perspektywa logiczna/funkcjonalna

Zakres funkcjonalny systemu obejmuje:

- konta użytkowników i role,
- planowanie oraz obsługę zajęć (sesje, zapisy, obecność),
- rozliczenia miesięczne oraz statusy opłat.

Warstwa UI znajduje się w SPA. Backend realizuje logikę biznesową, walidacje oraz kontrolę dostępu.

4.6 Perspektywa sprzętowa (deployment) i środowisko uruchomieniowe

Docelowe uruchomienie zakłada serwer z systemem Linux (np. Ubuntu/Debian) oraz konteneryzację (Docker Compose). Wystawiony na zewnątrz jest wyłącznie Nginx, natomiast backend i baza danych działają w sieci prywatnej.
Elementy środowiska uruchomieniowego:

- Nginx – reverse proxy oraz serwowanie plików SPA,

- Django API (WSGI, np. Gunicorn) – obsługa endpointów REST,
- PostgreSQL – baza danych uruchomiona wewnątrz sieci prywatnej.

4.7 System bazy danych i mechanizmy utrwalania danych

Utrwalanie danych realizowane jest w PostgreSQL (dane kont, zajęć, zapisów, obecności i rozliczeń). Dostęp do danych realizuje ORM Django.
Operacje utrzymaniowe:

- migracje schematu bazy (narzędzia Django),
- trwałe przechowywanie danych (wolumen DB),
- kopie zapasowe (zakres wdrożeniowy/operacyjny).

4.8 Raportowanie oraz analityka/BI

Raportowanie operacyjne opiera się na danych rozliczeniowych:

- generacja miesięcznych należności (`Purchase`),
- podgląd statusów opłat (np. lista zaległości na dany miesiąc).

Analityka/BI może zostać rozszerzona przez:

- eksport danych (np. CSV),
- dedykowane endpointy raportowe w panelu administracyjnym.

4.9 Mechanizmy zarządzania

Zarządzanie projektem i wdrożeniem obejmuje:

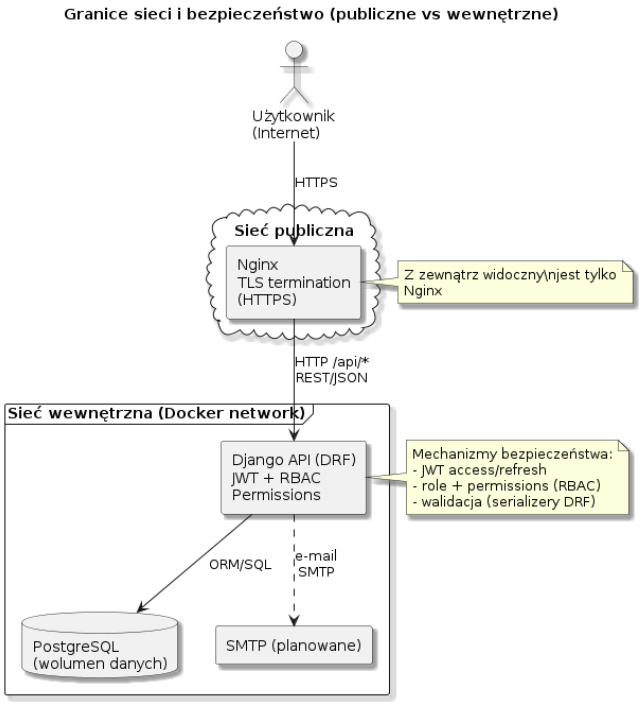
- CI (lint/test/build),
- proces wdrożeniowy wraz z migracjami bazy,
- logowanie operacyjne (logi aplikacji i kontenerów).

4.10 Mechanizmy bezpieczeństwa

Bezpieczeństwo systemu zapewniają:

- szyfrowanie komunikacji przez HTTPS (terminacja TLS na Nginx),
- autoryzacja przez JWT (access/refresh),
- kontrola dostępu oparta o role i uprawnienia (RBAC, permissions),
- walidacja danych wejściowych (serializery DRF) i ograniczenia modeli,

- izolacja komponentów: API i PostgreSQL w sieci prywatnej (brak bezpośredniej ekspozycji).



5 Dane trwałe

5.1 Model logiczny danych

Model danych został zaprojektowany w podejściu relacyjnym i obejmuje trzy główne obszary: konta i role użytkowników, obsługę zajęć (grupy, sesje, zapisy, obecności) oraz rozliczenia. Diagram ER przedstawia zależności pomiędzy encjami oraz kluczowe ograniczenia spójności (m.in. relacje wiele-do-jednego oraz ograniczenia unikalności).
Ze względu na rozmiar diagramu, w treści pracy zamieszczono link, pod którym znajduje się pełna wersja diagramu.(zaleca się pobrać, bo przy podglądzie może być gorszej jakośći)

Pobierz pełny diagram ER (PDF)

5.2 Przetwarzanie i przechowywanie danych

Dane systemu są przechowywane w relacyjnej bazie PostgreSQL. Dostęp do danych realizuje backend Django z wykorzystaniem ORM, co zapewnia spójne

mapowanie modeli domenowych na tabele oraz ułatwia utrzymanie schematu. Operacje na danych są wykonywane poprzez REST API (Django REST Framework). Walidacja danych wejściowych odbywa się na poziomie serializerów DRF oraz ograniczeń modeli, a kontrola dostępu do operacji jest realizowana przez JWT (access/refresh) oraz uprawnienia zależne od ról użytkownika.
Zmiany schematu bazy danych są wprowadzane poprzez migracje Django, wersjonowane w repozytorium. W bazie stosowane są indeksy i ograniczenia unikalności wspierające integralność danych (np. brak duplikatów zapisów na grupę dla tego samego ucznia). Nie zakłada się programowania logiki biznesowej bezpośrednio w bazie danych (np. wyzwalaczy); logika pozostaje po stronie backendu, co upraszcza rozwój i testowanie aplikacji.

6 Specyfikacja analityczna i projektowa

Repozytorium kodu – Gitlab

6.1 Metody realizacji projektu

Do napisania frontendu wykorzystaliśmy framework JavaScript'u - React. Jako backend został wykorzystany framework Pythona – Django. Dla łatwego uruchamiania i konteneryzacji został wykorzystany Docker Compose.
Żeby przyśpieszyć proces wdrażania i integracji, został wykorzystany mechanizm gitlab CI/CD. Na nim został napisany pipeline składający się z trzech jobów:

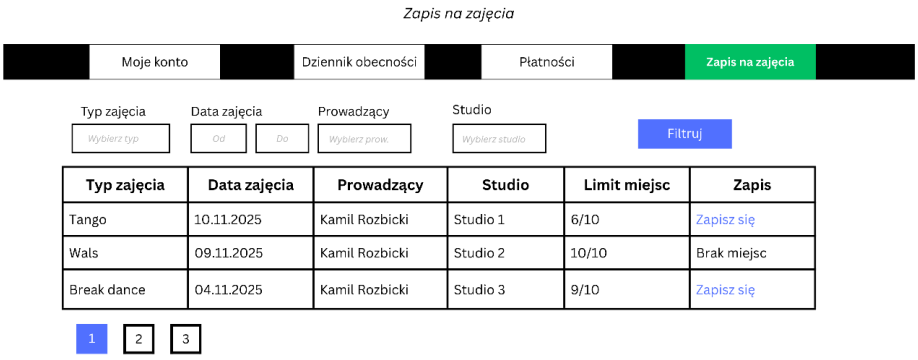
1. Budowanie frontendu
2. Testowanie i lintowanie frontendu
3. Testowanie i lintowanie backendu

Diagram ER można zobaczyć w załączniku 5.1. Diagram wdrożenia widać na rysunku 1. Są również diagramy sekwencji: rysunek 2 i rysunek 3.

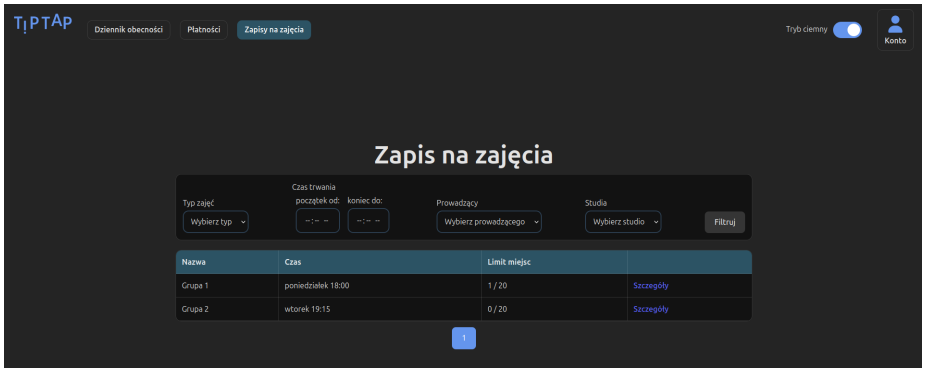
6.2 Statystyki

Nie licząc plików konfiguracyjnych oraz plików `.css`, wyszło

- 8500 linii kodu
- 110 plików
- 28 testów jednostkowych



Rysunek 4: Prototyp interfejsu



Rysunek 5: Wersja aktualna interfejsu

7 Projekt standardu interfejsu użytkownika

W centrum uwagi naszego designu została umieszczona prostota połączona z funkcjonalnością systemu. Głównie opieramy się na 3 szkicach persony w naszym projekcie – Student, Administrator oraz Instruktor. Stawiamy na dostępność, dlatego zrobiliśmy dla nich prosty wygląd stron.

7.1 Narzędzia

Aby zapewnić spójność naszych poglądów na, to jak będzie wyglądać nasz UI, stworzyliśmy dokument `pdf`, zawierający prototypy naszych stron w projekcie. Aby stworzyć ten dokument, wykorzystaliśmy narzędzie Canva.

7.2 Wpływ prototypu na gotowy produkt

Nawet jeśli produkt odbiegałby od prototypu, samo jego stworzenie miało wartość dla zespołu – ustaliliśmy wspólną wizję rozwiązania. Ponadto, obecny interfejs rzeczywiście przypomina ten prototypowy. Widać to, porównując rysunek 4 i rysunek 5.

8 Specyfikacja testów

Aplikację testowaliśmy zgodnie ze scenariuszami, które wyniknęły z przypadków użycia. Te testy były manualne i wykonywał je recenzent przed dopuszczeniem nowej zmiany do gałęzi głównej repozytorium. Przed prezentacją aplikacji ponownie upewniliśmy się, że wszystko działa.

8.1 Standardy jakości

Oprócz weryfikacji poprawności aplikacji w sensie logiki biznesowej powinnością recenzenta było, pilnowanie zgodności aplikacji z pewnymi standardami. Potrzebę spełnienia tych standardów narzucają wymagania **N1** i **N3**.

Urządzenia mobilne Interfejs powinien działać również na wąskich ekranach.

Dostępność Interfejs nie powinien generować błędów związanych z dostępnością, np. niski kontrast, niepodpisane przyciski.

8.2 Testy jednostkowe

Aby wcześnie wykrywać nowopowstałe błędy oraz przyspieszyć proces testowania, oraz polepszyć strukturę kodu korzystaliśmy z testów jednostkowych.
Użyliśmy ich zarówno na frontendzie, jak i na backendzie. Na backendzie testowaliśmy zachowanie endpointów. Przykładowo, poprawna prośba o zapisanie ucznia do grupy zajęciowej powinna skutkować pozytywną odpowiedzią serwera oraz utworzeniem odpowiednich wpisów w bazie danych.

8.2.1 Sytuacje wyjątkowe

Testy jednostkowe pozwalają również sprawdzić to, czego nie da się sprawdzić poprzez interfejs. W szczególności reakcję serwera na zapytanie złośliwie spreparowane z pominięciem interfejsu. Próba zarejestrowania do nieistniejącej grupy powinna skutkować komunikatem o niepowodzeniu, a nie błędem serwera.

8.2.2 Obsługa sytuacji wyjątkowych

Gdy zostanie wykryta sytuacja wyjątkowa, np. dane ucznia nie zawierają daty urodzenia, kod to wykrywa i zgłasza wyjątek. Serwer jest tak skonfigurowany, że ten błąd przekształca na odpowiedź, która mówi, co jest źle w którym polu formularza.
Taki komunikat wyświetlamy na czerwono pod odpowiednim polem formularza.

8.2.3 Automatyczne testy interfejsu

Pisząc niektóre komponenty interfejsu, wspomogliśmy się testami jednostkowymi. Przetestowaliśmy między innymi czy motyw kolorystyczny interfejsu zapisuje się między sesjami oraz, czy przy pierwszej wizycie dostosowuje się do preferencji zapisanych w przeglądarce.

8.2.4 Miary jakości testów

Nie korzystaliśmy z miary pokrycia kodu testami. Uznaliśmy, że biorąc pod uwagę nasze niskie doświadczenie, mogłoby to doprowadzić do pisania testów „pod" pokrycie, a nie takich, które są użyteczne.

9 Wirtualizacja / konteneryzacja

Użyliśmy kontenerów dockerowych, które symulują środowisko wdrożeniowe, przybliżając je do programisty. Jednocześnie wymusza to, aby cała konfiguracja istotna z punktu widzenia wdrożenia była zawarta w repozytorium. Niweluje to szansę wystąpienia sytuacji, w której to, że aplikacja działa, jest efektem jakiejś niejawnej specyfiki/konfiguracji komputera programisty.

10 Bezpieczeństwo

Bezpieczeństwo danych oparte jest na identyfikatorach sesji, które są tak zwanymi „secure cookies". Serwer korzysta wyłącznie z połączeń szyfrowanych. Z tych powodów, poufność identyfikatora jest odporna na ataki XSS oraz man-in-the-middle. W praktyce pozostaje bezpieczeństwo hasła oraz złośliwe oprogramowanie, które mogło zainfekować przeglądarkę, ale to leży w gestii użytkownika.

10.1 Uprawnienia użytkowników

Każdy użytkownik ma przypisaną jedną z trzech ról: uczeń, instruktor albo admin. Jest jeszcze superadmin, ale on może się zalogować tylko z sieci prywatnej między kontenerami.

Role są przydzielane do kont, a konto jest związane z identyfikatorem sesji, który otrzymuje użytkownik po zalogowaniu się, i który następnie służy do identyfikacji użytkownika. Tak więc, złamanie ról wymaga podszycia się pod użytkownika z daną rolą, co, jak wyjaśniliśmy powyżej, jest bardzo trudne, nie znając jego hasła.
Role są wykorzystywane do określenia czy dany użytkownik ma dostęp do danej operacji. Np. tylko administrator może dodać grupę zajęciową, użytkownik ma dostęp jedynie do odczytu.

11 Podręcznik użytkownika

11.1 Uczeń

Jak wspomnieliśmy w sekcji 8.1, interfejs był pisany zgodnie ze standardami dostępności. Ponadto, uczeń szkoły TipTap nie będzie miał zamiaru zapoznawać się instrukcją, tylko po to, żeby wykonać czynność, która jest oczywista.

Jedyna wiedza, jakiej uczeń potrzebuje, to znajomość reguł szkoły, np. tego, że zapisuje się do grupy zajęciowej, a nie na pojedyncze zajęcia, albo to, że co miesiąc winien jest opłacić swój karnet. To zadanie spełnia strona wizytówkowa i załączony tam regulamin.

Z tych powodów wierzymy, że instrukcja jest niepotrzebna.

11.2 Instruktor

Instruktor ma jeszcze mniej opcji niż uczeń, komentarz jest zbędny.

11.3 Administrator

Jedyną osobą, której winni jesteśmy instrukcje, jest właścicielka. Ma ona trochę okienek do wyboru i całość może nie być intuicyjna. Niemniej jednak widzieliśmy się z nią widzieć podczas oddania projektu, a z ludzkiej rozmowy można wynieść więcej niż ze stosu instrukcji, stąd ponownie pomijamy instrukcje.

12 Podręcznik administratora

12.1 0) Informacje ogólne o systemie

System składa się z trzech usług uruchamianych w Docker Compose:

- **db** – PostgreSQL 16 (trwałe dane w wolumenie `db-data`)
- **backend** – aplikacja Django + DRF uruchamiana przez Gunicorn na porcie `8000`

- **frontend** – aplikacja SPA (React/Vite) serwowana przez Nginx na portach `80/443`, z proxy do backendu pod `/api/`

Dostępne punkty kontrolne:

- `GET /api/ping/` – zwraca `{"status":"ok"}` (używane także w healthcheck backendu)

  Uwaga: `APPEND_SLASH = False`, więc endpointy trzeba wywoływać dokładnie z takim "/", jak w routingu (np. `/api/ping/`, a nie `/api/ping`).

12.2 1) Instrukcja budowy systemu z kodu źródłowego

12.2.1 1.1 Wariant zalecany: Docker Compose

Wymagania: Docker + Docker Compose (v2).

1. Wejdź do katalogu głównego projektu (tam, gdzie jest `docker-compose.yml`).
2. Zbuduj obrazy:

   ```
   docker compose build
   ```

3. Uruchom usługi w tle:

   ```
   docker compose up -d
   ```

4. Sprawdź status:

   ```
   docker compose ps
   ```

5. Sprawdź działanie backendu:

   ```
   curl -fsS http://localhost:8000/api/ping/
   ```

Porty:

- Backend: `http://localhost:8000/`
- Frontend: `http://localhost/` (przekierowanie na HTTPS) oraz `https://localhost/`

12.2.2 1.2 Uruchomienie lokalne (bez Dockera) – głównie do developmentu

Backend:

1. Python 3.12+
2. Instalacja zależności:

```
python -m venv .venv
# Linux/Mac:
source .venv/bin/activate
# Windows:
# .venv\Scripts\activate
pip install -r requirements.txt
```

3. Ustaw zmienne środowiskowe (opis w rozdziale 2) i uruchom:

```
python manage.py migrate
python manage.py runserver 0.0.0.0:8000
```

Frontend (Vite):

- W konfiguracji developerskiej serwer działa na porcie `3000` i proxy'uje `/api` do `http://localhost:8000`.

```
npm install
npm run dev
```

12.3 2) Instrukcja instalacji i konfiguracji systemu

12.3.1 2.1 Konfiguracja backendu (Django)

Backend konfigurowany jest przez zmienne środowiskowe:

- `DJANGO_DEBUG` – `0/false/no` wyłącza debug (zalecane w produkcji)
- `DJANGO_SECRET_KEY` – klucz sekretu Django (w produkcji musi być unikalny i tajny)
- `DJANGO_ALLOWED_HOSTS` – lista hostów po przecinku (np. `backend,localhost,127.0.0.1,example.com`)
- `CORS_ALLOWED_ORIGINS` – dozwolone originy dla frontendu (np. `http://localhost:3000,https://localhost`)
- `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_HOST`, `POSTGRES_PORT` – dane dostępowe do bazy

Sekrety (zalecane w produkcji): Projekt wspiera czytanie sekretów z `/run/secrets/<nazwa>`. Dla bazy danych może to być np. `db_app_passwd` – wtedy hasło nie musi być w `.env` ani w compose.
Baza danych:

- Jeśli ustawione jest `POSTGRES_DB`, backend użyje PostgreSQL.
- Jeśli nie, przełączy się na SQLite (tryb developerski).

Statyczne pliki:

- Podczas budowania obrazu backendu wykonywane jest `collectstatic`, a w runtime statyki obsługuje WhiteNoise.

12.3.2 2.2 Konfiguracja reverse proxy (Nginx) i HTTPS

Frontend jest serwowany przez Nginx:

- port `80` przekierowuje na `443` (HTTP → HTTPS)
- na `443` serwowany jest frontend SPA oraz proxy do backendu:
  - `/api/` → `http://backend:8000`
  - SPA fallback: wejście na dowolny route (np. `/payments`, `/classes/...`) zwróci `/index.html`

Certyfikaty: Nginx oczekuje certyfikatów w kontenerze pod:

- `/etc/nginx/ssl/cert.pem`
- `/etc/nginx/ssl/priv.pem`

W środowisku produkcyjnym należy zapewnić prawidłowe certyfikaty (np. Let's Encrypt) i podmontować je w te ścieżki.

> **Uwaga:** w tej konfiguracji Nginx zwraca `404` dla `/api/admin`. Panel Django admin jest dostępny bezpośrednio na backendzie (`http://localhost:8000/admin/`), o ile port backendu jest wystawiony.

12.3.3 2.3 Konfiguracja frontendu (API URL)

Frontend korzysta z `VITE_API_URL` jako bazowego adresu API:

- Jeśli `VITE_API_URL` nie jest ustawione, domyślnie używane jest `https://localhost`.
- W trybie produkcyjnym najprościej ustawić `VITE_API_URL` na ten sam host, z którego serwowany jest frontend (np. `https://twoja-domena.pl`), ponieważ Nginx i tak proxy'uje `/api/`.

12.4 3) Instrukcja aktualizacji oprogramowania

12.4.1 3.1 Aktualizacja w Docker Compose (typowa procedura)

1. Zaktualizuj kod źródłowy (np. `git pull`).
2. Przebuduj i uruchom ponownie:

   ```
   docker compose up -d --build
   ```

3. Migracje bazy danych:
   - backend wykonuje `python manage.py migrate -noinput` automatycznie w `entrypoint.sh`,
   - jednak przy większych zmianach zaleca się wykonać migracje świadomie:

     ```
     docker compose exec backend python manage.py migrate
     ```

12.4.2 3.2 Aktualizacja frontendu

Frontend jest dostarczany jako statyczny build w obrazie kontenera `frontend`. Aktualizacja to przebudowanie obrazu i restart (w praktyce realizuje to `docker compose up -d -build`).

12.4.3 3.3 Rollback (procedura awaryjna – skrót)

- Jeśli wdrożenie powoduje błędy, wróć do poprzedniego obrazu (tag/commit) i uruchom ponownie kontenery.
- Jeśli problem dotyczy migracji DB, rollback może wymagać przywrócenia kopii bazy (rozdz. 5).

12.5 4) Instrukcja zarządzania użytkownikami i uprawnieniami

12.5.1 4.1 Panel administracyjny Django

Panel admina:

- `http://localhost:8000/admin/`

Utworzenie superusera:

```
# lokalnie
python manage.py createsuperuser

# w Dockerze
docker compose exec backend python manage.py createsuperuser
```

12.5.2 4.2 Role i logika uprawnień

W systemie występują role (co najmniej): `student`, `instructor`, `admin`. Uprawnienia API są sprawdzane na podstawie:

- roli użytkownika (np. dostęp admina),
- profilu powiązanego (`student` / `instructor`) w wybranych funkcjach.

Ważne: Role muszą istnieć w bazie danych (np. utworzone przez seed/migrację lub ręcznie w panelu). Jeśli rola nie istnieje, część operacji administracyjnych zwróci błąd.
Przykładowe zasady (skrót):

- Student: zapisy/wycofanie z grup, podgląd własnych danych i własnej frekwencji
- Instruktor: podgląd uczestników i uzupełnianie obecności dla przypisanych zajęć
- Admin: pełen dostęp do zarządzania użytkownikami, grupami i sesjami, płatnościami i produktami

12.5.3 4.3 Uwierzytelnianie (JWT)

Frontend używa JWT:

- `POST /api/accounts/token/` – logowanie (email + password)
- `POST /api/accounts/token/refresh/` – odświeżanie tokenu
- `POST /api/accounts/register/` – rejestracja
- `GET /api/accounts/me/` – dane zalogowanego użytkownika

Token access jest przesyłany w nagłówku: `Authorization: Bearer <token>`.

12.5.4 4.4 Operacje administracyjne na kontach (przykłady)

W API są dostępne funkcje administracyjne:

- przegląd kont, filtrowanie, dezaktywacja (w praktyce ustawienie `is_active = false`)
- tworzenie studentów i instruktorów przez admina (z hasłem i danymi profilu)

  Rekomendacja: dezaktywacja kont (soft delete) jest bezpieczniejsza niż fizyczne usuwanie rekordów, bo nie zrywa powiązań (np. obecności, płatności).

12.6 5) Instrukcja tworzenia kopii zapasowych i odtwarzania systemu

12.6.1 5.1 Backup bazy danych (PostgreSQL)

Metoda 1: pg_dump (zalecana)

```
docker compose exec db pg_dump -U app szkola > backup.sql
```

Odtwarzanie:

1. (Opcjonalnie) zatrzymaj backend, żeby nie pisał do DB w trakcie importu:

   ```
   docker compose stop backend
   ```

2. Zaimportuj dump:

   ```
   cat backup.sql | docker compose exec -T db psql -U app szkola
   ```

3. Uruchom backend:

   ```
   docker compose start backend
   ```

12.6.2 5.2 Backup wolumenu (alternatywnie)

Dane Postgresa są w wolumenie `db-data`. W środowiskach produkcyjnych można wykonywać snapshot wolumenu (zależnie od infrastruktury).

12.6.3 5.3 Backup konfiguracji i certyfikatów

Do odtworzenia środowiska produkcyjnego ważne są również:

- certyfikaty TLS (pliki podmontowane do `/etc/nginx/ssl/...`)
- pliki `.env` / konfiguracja secretów (`/run/secrets/...`)

12.7 6) Instrukcja zarządzania zasobami systemu

12.7.1 6.1 Operacje na usługach (Docker)

Status usług:

```
docker compose ps
```

Logi:

```
docker compose logs -f backend
docker compose logs -f db
docker compose logs -f frontend
```

Restart pojedynczej usługi:

```
docker compose restart backend
```

Zatrzymanie całości:

```
docker compose down
```

12.7.2 6.2 Monitorowanie dostępności (healthcheck)

Backend posiada healthcheck oparty o:

- `GET http://localhost:8000/api/ping/`

W razie problemów:

- sprawdź logi backendu i połączenie z DB,
- upewnij się, że DB jest "healthy" (compose ma healthcheck `pg_isready`).

12.7.3 6.3 Wydajność i limity

- Gunicorn uruchamiany jest z `-workers 2` (konfiguracja w `entrypoint.sh` / `gunicorn.conf.py`). W środowisku produkcyjnym można zwiększyć liczbę workerów zależnie od CPU i obciążenia.
- Paginacja w wielu listach API jest włączona (domyślnie `page_size=10`, max `100` poprzez `page_size`).

12.7.4 6.4 Kluczowe endpointy operacyjne (skrót)

- System:
  - `GET /api/ping/`
- Szkoła (przykłady):
  - `GET /api/school/class-filters/`, `GET /api/school/classes/`
  - `POST /api/school/enroll/`, `POST /api/school/unenroll/`
  - `GET/POST /api/school/classes/<sessionId>/participants/` (obecności)
  - `CRUD /api/school/admin-sessions/` (tworzenie/edycja sesji przez admina)
- Billing:
  - `CRUD /api/billing/pass-products/` (produkty/karnety)
  - `CRUD /api/billing/purchases/` (należności/płatności)
  - `POST /api/billing/purchases/<id>/mark-paid/`
  - `POST /api/billing/purchases/<id>/void/`
  - `POST /api/billing/purchases/generate-monthly/` z payloadem `{"month":"YYYY-MM-01"}`

Checklista wdrożeniowa (krótka)

1. Ustaw `DJANGO_DEBUG=0`, `DJANGO_SECRET_KEY` (unikalny), `DJANGO_ALLOWED_HOSTS`.
2. Zmień hasła do bazy / przenieś do sekretów.
3. Zapewnij certyfikaty TLS dla Nginx.
4. Uruchom `docker compose up -d -build`.
5. Sprawdź `GET /api/ping/`.
6. Utwórz superusera i upewnij się, że role (`student/instructor/admin`) istnieją w DB.

13 Podsumowanie

Udało się spełnić kluczowe wymagania. Podczas prezentowania projektu właścicielce otrzymaliśmy kilka drobnych uwag oraz jedną, której zastosowanie byłoby nieco trudniejsze, wymaga ona zmiany modelu danych, choć nie jest to tak bolesne, jak mogłoby być, bo prawdziwych danych jeszcze w bazie nie ma. Należało wcześniej skonsultować nasze wątpliwości, zamiast implementować coś, co wydawało się sensowne.

13.1 Czego się nauczyliśmy?

Uważamy, że poczyniliśmy postępy w umiejętności pracy zespołowej.
W szczególności zauważyliśmy, że niektóre zadania, które wcześniej próbowalibyśmy wykonać sami, dają się podzielić z członkiem zespołu – kluczem jest rozmowa. Nauczyliśmy się też, że wystarczy zadania umieszczać w ustalonym, widocznym miejscu, a znajdzie się wolontariusz, które się ich podejmie.

13.2 Dalsze kroki

Z racji, że programiści mieli do dyspozycji środowisko przypominające środowisko wdrożeniowe, wdrożenie nie powinno wymagać dużo pracy. Aplikacja nie nadaje się jeszcze do wdrożenia, bo Właścicielka miała kilka uwag.

Zatwierdzam dokumentację.	..... Data i podpis Mentora